

[Dashboard](#) / [My courses](#) / [COMS3005A-AAA-2022](#) / [Class Test](#) / [2021 Class Test \(including practical question\)](#) / [Preview](#)

You can preview this quiz, but if this were a real attempt, you would be blocked because:

This quiz is not currently available

Question **1**

Not yet answered

Marked out of 2.00

Let $g(n)$ be the total cost so far from the root node to node n , and let $h(n)$ be the heuristic estimate from node n to the goal. Select the statements below that are true

- ☐ a. If we implement A* search with $h(n) = 0$, then our algorithm reduces to greedy best first search
- ☐ b. If we implement A* search with $h(n) = -g(n)$, then our algorithm reduces to greedy best first search
- ☐ c. If we implement A* search with $h(n) = 0$, then our algorithm reduces to uniform cost search
- ☐ d. If we implement A* search with $h(n) = 0$, then our algorithm reduces to breadth first search

Question 2

Not yet answered

Marked out of 2.00

Consider the Quicksort algorithm when the pivot is selected to be the rightmost element. That is, the Quicksort algorithm presented below:

Algorithm 10 quickSort(*left*, *right*)

The quicksort algorithm

Input: *left*, *right* where *left* and *right* are indices into an array of n entries (initially indexed $0 \dots n - 1$)

Output: a portion of the array *myList* where the values in *myList* are such that $myList[left] \leq myList[left + 1] \leq \dots \leq myList[right - 1] \leq myList[right]$

```

01 IF right > left
02   Then
03      $v \leftarrow myList[right]$ 
04      $i \leftarrow left$ 
05      $j \leftarrow right$ 
06     While  $i < j$ 
07       While  $myList[i] < v$ 
08          $i \leftarrow i + 1$ 
09       While  $j > i$  and  $myList[j] \geq v$ 
10          $j \leftarrow j - 1$ 
11       If  $j > i$ 
12         Then
13            $t \leftarrow myList[i]$ 
14            $myList[i] \leftarrow myList[j]$ 
15            $myList[j] \leftarrow t$ 
16         ELSE  $t \leftarrow myList[i]$ 
17            $myList[i] \leftarrow myList[right]$ 
18            $myList[right] \leftarrow t$ 
19       quickSort(left,  $i - 1$ )
20       quickSort( $i + 1$ , right)
  
```

From the options below, list all the kinds of input arrays that would lead to the **worst case** performance of the algorithm?

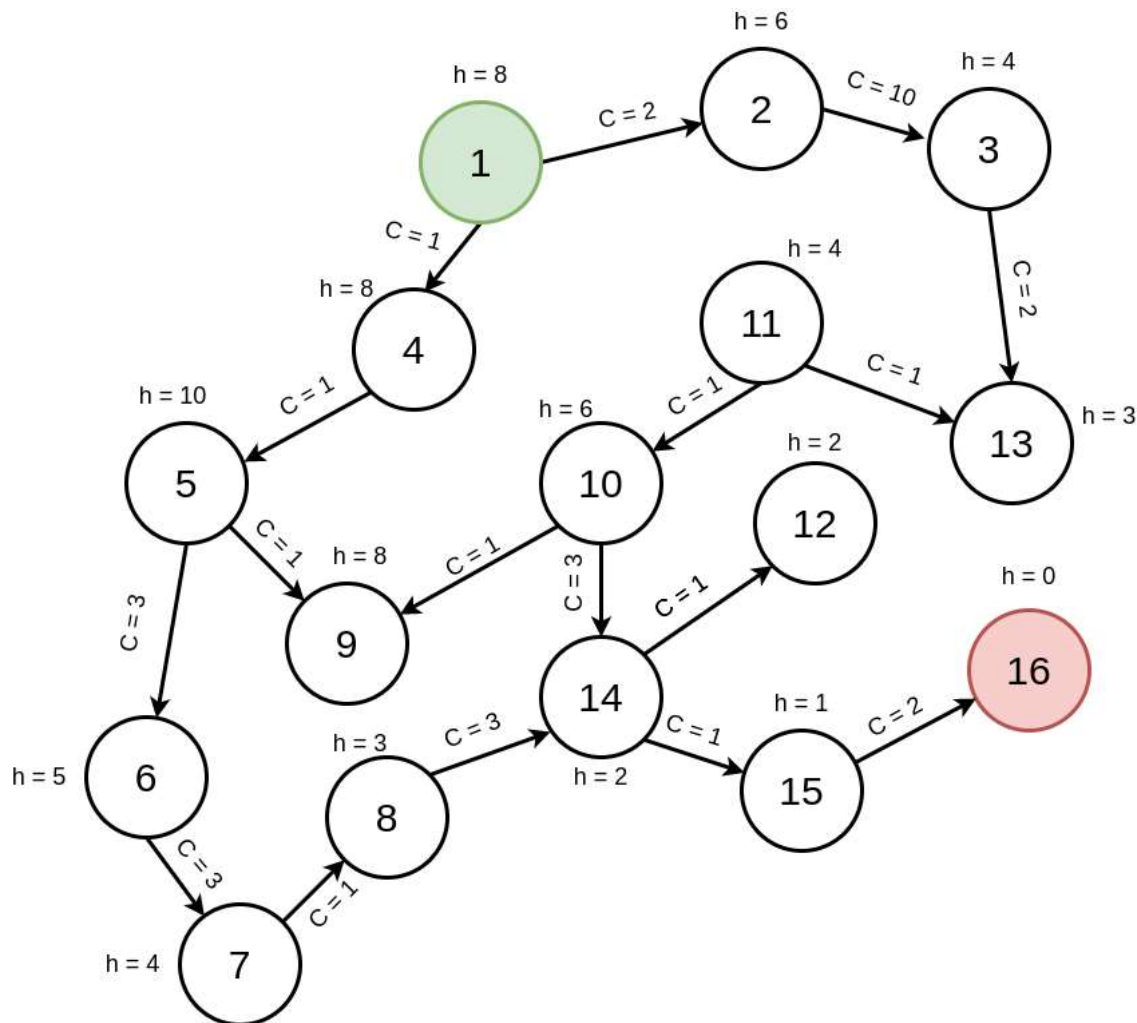
- ☐ a. The input array is randomly ordered
- ☐ b. The input array is in reverse sorted order
- ☐ c. The input array is in sorted order
- ☐ d. The first half of the input array is sorted, while the second half is in reverse sorted order.
- ☐ e. Every element of the input array is identical

Question 3

Not yet answered

Marked out of 2.00

Let C be the cost of traversing an edge between nodes, and let h be the heuristic estimate of a given node to the goal. Consider the graph below, where nodes are labelled with integers and directed edges are indicated by lines with arrows. Each edge is labelled with a cost C , and each node with a heuristic value h .



Assume that we wish to search for the goal node 16 (in red) starting at the node 1 (in green) using the exact **breadth first search** algorithm below. Further, assume that when multiple child nodes are added to the queue, they are added in ascending order of node label (e.g. if there are three children 1, 2, 3, then they are added in exactly that order).

function BREADTH-FIRST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

frontier $\leftarrow$ a FIFO queue with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the shallowest node in *frontier* */

add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

if *problem*.GOAL-TEST(*child*.STATE) **then return** SOLUTION(*child*)

frontier $\leftarrow$ INSERT(*child*, *frontier*)

In the answer box below, list the nodes in the order that they are added to the frontier by BFS until the algorithm terminates. **Be sure to separate each node by a comma (e.g. your answer would look like 1,2,3,4,...).** Note that not all nodes may be added to the frontier.

Answer:

Question 4

Not yet answered

Marked out of 2.00

Bob has created a new algorithm for robot path planning! Given a start and goal location, his algorithm determines a path that begins at the start location and reaches the goal. If the goal is reachable from the start, Bob's algorithm will always return a path; however, it may not be the shortest possible path.

Select the statement below that is true.

- ☐ a. Bob's algorithm is admissible
- ☐ b. Bob's algorithm is optimal
- ☐ c. Bob's algorithm is consistent
- ☐ d. Bob's algorithm is complete.

Question 5

Not yet answered

Marked out of 2.00

A sorting algorithm is said to be "in-place" if it does not use extra space for manipulating the input, but may require a small though nonconstant extra space for its operation. Select all of the algorithms below that are "in-place".

- ☐ a. Quicksort
- ☐ b. Mergesort
- ☐ c. Selection sort
- ☐ d. Bubblesort
- ☐ e. Insertion sort

Question 6

Not yet answered

Marked out of 1.00

Assume we are tackling a problem that is known to have the optimal substructure property. Select the statement below that is true

- ☐ a. We can guarantee that a greedy approach will produce the optimal solution
- ☐ b. We need more information before we can determine whether a greedy approach will produce the optimal solution
- ☐ c. We can guarantee that a greedy approach will not produce the optimal solution

Question 7

Not yet answered

Marked out of 1.00

Consider the 8-puzzle problem discussed in class. The heuristic $h(n) = 0$ is admissible. True or false?

Select one:

- ☐ True
- ☐ False

Question 8

Not yet answered

Marked out of 1.00

Consider a problem where zero or negative step costs are allowed. In this case, breadth-first search is still complete. True or false?

Select one:

- ☐ True
- ☐ False

Question 9

Not yet answered

Not graded

The practical question is worth 10% of the overall mark - I recommend leaving this question for the very end.

Instructions:

1. Read the question below and submit your solution [here](#)
2. **Do not submit this Moodle quiz until you have submitted your solution to the practical question. If you do this, the practical submission link will become unavailable.**

[Click HERE to download the PDF of the question](#)

[illegible]

Question **10**

Not yet answered

Marked out of 2.00

Select all of the statements below that are true.

- ☐ a. It is impossible to have a heuristic that is both admissible and consistent. It is either one or the other.
- ☐ b. Any admissible heuristic is always consistent.
- ☐ c. It is possible to have a consistent heuristic that is not admissible.
- ☐ d. It is possible to have an admissible heuristic that is not consistent.
- ☐ e. Any consistent heuristic is always admissible.

Question **11**

Not yet answered

Marked out of 2.00

Assume we implement a greedy approach to the making change problem in an attempt to make change for 15 cents. Select all of the coin sets below that will not result in an optimal solution when the greedy algorithm is used.

- ☐ a. 1, 2, 5, 15, 25
- ☐ b. 1, 2, 7, 10
- ☐ c. 1, 2, 5, 10, 25
- ☐ d. 1, 2, 5, 10, 15

Question **12**

Not yet answered

Marked out of 2.00

Assume that we wish to sort an array, but we know that the array is either already sorted, or very nearly sorted. Which of the algorithms below would be the most preferred choice of sort?

- ☐ a. Quicksort
- ☐ b. Mergesort
- ☐ c. Selection sort
- ☐ d. Insertion sort

Question **13**

Not yet answered

Marked out of 1.00

Assume we are operating on a computer with very little memory, and we are tackling a problem with the following properties:

1. Step costs may be zero or negative
2. The maximum search depth is infinite

Select the best algorithm to use in this situation.

- ☐ a. Breadth first search
- ☐ b. Uniform cost search
- ☐ c. Depth first search

Question **14**

Not yet answered

Marked out of 1.00

What is the theoretical lower bound on the number of comparisons performed by a comparison-based sorting algorithm?

- ☐ a. $\Omega(n)$
- ☐ b. $\Omega(n^2)$
- ☐ c. $\Omega(2^n)$
- ☐ d. $\Omega(1)$
- ☐ e. $\Omega(\lg n)$
- ☐ f. $\Omega(n \lg n)$

Question **15**

Not yet answered

Marked out of 2.00

Assume we are operating on a computer with very little memory, and we are tackling a problem with the following properties:

1. All step costs are identical
2. The maximum search depth is infinite

Select the best algorithm to use in this situation.

- ☐ a. Depth first search
- ☐ b. Uniform cost search
- ☐ c. Iterative deepening search
- ☐ d. Breadth first search

Question 16

Not yet answered

Marked out of 2.00

Consider the algorithm in the pseudocode below

algorithm(*myList*, *n*)

Input: myList, n where *myList* is an array with *n* entries (indexed $0 \dots (n - 1)$)

Output: myList

00 *max* $\leftarrow$ maximum number in *myList*

01 For *i* from 0 to *max*

02 *h*[*i*] $\leftarrow$ 0

03 For *i* from 0 to *n* - 1

04 *t* $\leftarrow$ *myList*[*i*]

05 *h*[*t*] $\leftarrow$ *h*[*t*] + 1

06 *i* = 0

07 For *j* from 0 to *max*

08 If *h*[*j*] > 0

09 For *k* from 1 to *h*[*j*]

10 *myList*[*i*] $\leftarrow$ *j*

11 *i* $\leftarrow$ *i* + 1

12 Return *myList*

What is the space and time complexity of the algorithm?

- ☐ a. Space complexity: $O(n + m)$
Time complexity: $O(m)$

where *m* is the maximum value in the array

- ☐ b. Space complexity: $O(m)$
Time complexity: $O(m)$

where *m* is the maximum value in the array

- ☐ c. Space complexity: $O(n)$
Time complexity: $O(n)$

- ☐ d. Space complexity: $O(n)$
Time complexity: $O(n \lg n)$

where *m* is the maximum value in the array

- ☐ e. Space complexity: $O(n)$
Time complexity: $O(n \lg n + m)$

where *m* is the maximum value in the array

- f. Space complexity: $O(m)$
Time complexity: $O(n + m)$

where m is the maximum value in the array

- ☐ g. Space complexity: $O(n + m)$
Time complexity: $O(n + m)$

where m is the maximum value in the array

- ☐ h. Space complexity: $O(m)$
Time complexity: $O(m \lg m)$

where m is the maximum value in the array

[← Exam Practical Submission \(hidden\)](#)

Jump to...

[Class Test Practical Submission \(hidden\) ►](#)